A High-Assurance Evaluator for Machine-Checked Secure Multiparty Computation

Karim Eldefrawy
SRI International

Vitor Pereira
INESC TEC and FC Universidade do Porto

ABSTRACT

Secure Multiparty Computation (MPC) enables a group of n distrusting parties to jointly compute a function using private inputs. MPC guarantees correctness of computation and confidentiality of inputs if no more than a threshold t of the parties are corrupted. Proactive MPC (PMPC) addresses the stronger threat model of a *mobile adversary* that controls a changing set of parties (but only up to t at any instant), and may eventually corrupt *all n parties* over a long time.

This paper takes a first stab at developing high-assurance implementations of (P)MPC. We formalize in EasyCrypt, a tool-assisted framework for building high-confidence cryptographic proofs, several abstract and reusable variations of secret sharing and of (P)MPC protocols building on them. Using those, we prove a series of abstract theorems for the proactive setting. We implement and perform computer-checked security proofs of concrete instantiations of the required (abstract) protocols in EasyCrypt.

We also develop a new tool-chain to extract high-assurance executable implementations of protocols formalized and verified in EasyCrypt. Our tool-chain uses Why3 as an intermediate tool, and enables us to extract executable code from our (P)MPC formalizations. We conduct an evaluation of the extracted executables by comparing their performance to performance of manually implemented versions using Python-based Charm framework for prototyping cryptographic schemes. We argue that the small overhead of our high-assurance executables is a reasonable price to pay for the increased confidence about their correctness and security.

CCS CONCEPTS

• **Security and privacy → Logic and verification**; *Formal security models*; • **Software and its engineering →** *Source code generation.*

KEYWORDS

Secure Multiparty Computation, Verified Implementation, High-Assurance Cryptography

CCS '19, November 11–15, 2019, London, United Kingdom

ACM ISBN 978-1-4503-6747-9/19/11…$15.00
https://doi.org/10.1145/3319535.3354205

ACM Reference Format:
Karim Eldefrawy and Vitor Pereira. 2019. A High-Assurance Evaluator for Machine-Checked Secure Multiparty Computation. In *2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19), November 11–15, 2019, London, United Kingdom.* ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/3319535.3354205

1 INTRODUCTION

Correctly designing secure cryptographic primitives and protocols is a non-trivial task. We argue that not only is it hard to correctly design them, but so is it to correctly and securely implement them in software. This issue is particularly amplified in settings where protocols involve multiple parties, and when they should guarantee security against strong adversaries beyond passive ones[1].

There are several efforts implementing (advanced) cryptographic primitives and protocols available to developers, such as OpenSSL [2], s2n [3], BouncyCastle [4], Charm [2], SCAPI [32], FRESCO [5] [28], TASTY [47], SCALE-MAMBA [6], EMP [7], Sharemind [20, 21]. Such tools and libraries typically aim to improve usability, software reliability, and performance. Some of them also target cloud-based and large distributed applications, aiming to deploy secure and privacy-preserving distributed computations to address practical challenges. A missing aspect of most of these efforts is the increased confidence obtained in security and correctness of the design and implementation of such complex (cryptographic) algorithms and protocols when utilizing computer-aided formal verification and synthesis. Because of this, subsequent work [3, 4, 9, 13, 17, 18, 22, 39] started tackling verification of cryptographic primitives and protocols, and software implementations thereof. One notable example is Project Everest [8] which provides a collection of tools and libraries that can be combined together and generate a mixture of C and assembly code that implements TLS 1.3, with proofs of safety, correctness, security and various forms of side-channel resistance.

Several authors began exploring how such high-assurance design and implementation of cryptography can be applied to more complex secure (two-party) computation and multiparty computation (MPC) protocols [4, 8, 21, 26, 27, 45, 54, 62]. Nevertheless, we identify a series of limitations of such efforts:

1) *Limited number of parties* - [4, 27, 54] such work typically considers the case of two-party computation.

[1] Often called honest-but-curious or semi-honest, we use those terms interchangeably in this paper.
[2] https://www.openssl.org
[3] https://github.com/awslabs/s2n
[4] https://www.bouncycastle.org
[5] https://github.com/aicis/fresco
[6] https://github.com/KULeuven-COSIC/SCALE-MAMBA
[7] https://github.com/emp-toolkit
[8] https://project-everest.github.io

2) *Lack of security guarantees against strong adversaries* - the work that goes beyond two parties [8, 21, 44, 62, 66] typically focuses on the semi-honest adversary model.
3) *Lack of high-assurance implementations for active adversaries* - even when active adversaries and multiple parties are considered [44, 45], there are no automatically (and verifiably) synthesized executable implementations from protocol specifications that were checked using computer-aided verification.

Table 1 summarizes the most relevant recent work on verification of complex cryptographic protocols involving multiple parties and withstanding strong adversaries. A more detailed comparison with related work is provided in Section 6.

Finally, the semi-honest and active adversary models, while covering a large set of possible applications of MPC, do not cover scenarios in which complex distributed systems are built and run for long durations, and where strong persistent adversaries continuously attack them. For example, those two models do not cover the case where adversaries can corrupt all parties over a long period of time. Proactive MPC (PMPC) addresses the stronger threat model of a *mobile adversary* that controls a changing set of parties (but only up to t at any instant), and may eventually corrupt *all n parties* over a long time throughout the course of a protocol's execution, or lifetime of confidential inputs. The main intuition behind proactive secret sharing (that typically underlies PMPC) is to periodically re-randomize (*refresh*) shares of secrets and delete old ones, thus preventing an adversary that collects all shares from different periods to combine them together. In the proactive setting, parties are periodically reset (*recovered*) to a clean state to ensure that adversarial corruptions are purged. Such reset parties have to run a recovery protocol to obtain their new non-corrupted state (secret shares) and re-join the secure computation system.

To the best of our knowledge there are currently no publicly available high-assurance implementations of formally verified and machine-checked (P)MPC withstanding active adversaries. By high-assurance we mean automatically (and verifiably) synthesized from protocol specifications that were checked using computer-aided verification. This paper takes a first stab at developing such high-assurance implementations of (P)MPC, and as a side contribution performs the first computer-aided verification of a variant of the fundamental BGW [16] MPC protocol for passive and static active adversaries using EasyCrypt (with computational security in the latter case).

Below we discuss challenges facing our work, followed by details of our contributions addressing these challenges. We finish this section with a brief description of our final verified (P)MPC evaluator (illustrated in Figure 1).

Previous Work	Protocol / Num. Parties	Adversary Model	E2E Proof	High-Assurance
[66]	PCR [66] / 3 Parties	Passive	Yes	No
[4]	Yao [70] / 2 Parties	Passive	Yes	Yes
[44]	Maurer [56] / N Parties	Active	No*	No
Our Work	*BGW [16] / N Parties*	*Passive & (Pro)Active*	*Yes*	*Yes*

Table 1: Comparison of the most relevant computer-aided verification and high-assurance implementations of secure computation protocols. Private Count Retrieval (PCR) is an application-specific database querying protocol involving 3 parties (client, server, and a trusted third party). *The proof in [44] is not completely formalized and end-to-end (E2E) checked in EasyCrypt, only a proof that certain properties are satisfied by the protocol in [56] is checked in EasyCrypt, then it is manually proven that a simulator exists if these properties hold. Our work contains E2E proofs in EasyCrypt without any manual steps.

1.1 Challenges

We face two main challenges: developing machine-checked specifications and proofs for (P)MPC and underlying building block primitives, and obtaining automatically synthesized verified implementations of such protocols.

Machine-checked formalization and security proofs of MPC protocols (with multiple parties, sub-protocols, and guaranteeing security against mobile active adversaries) is a complex task that involves knowledge that spans cryptography, distributed computing and programming languages. Particularly, using a tool like EasyCrypt to come up with a machine-checked proof is not easy, because one has to accommodate complex cryptographic schemes and protocol using descriptions that can be used inside the EasyCrypt system. Additionally, formalizing MPC protocols also involves formalizing the underlying mathematical structures upon which they build. Despite EasyCrypt already providing a large set of mathematical constructions, we still needed to formalize additional libraries, specially to deal with polynomials over finite fields.

There are currently no (publicly available) tools for automatically synthesizing verified implementations of complex cryptographic (multiparty) protocols. Two possible approaches to tackle this objective consist on either starting with a software implementation of the protocol and then attempt to prove properties surrounding that implementation or starting with a proof script (with a formal specification and proofs) and derive a concrete implementation from it. In this work, we generated a concrete verified *correct-by-construction* executable software implementation from a proof script. Using our new EasyCrypt extraction tool-chain, we were able to obtain such implementation of an MPC evaluator with (optional) proactive security guarantees.

1.2 Contributions

In this paper, *we develop a high-assurance formally-verified proactive secure multiparty computation, (P)MPC, evaluator based on machine-checked proactive (verifiable) secret sharing and the BGW [16] protocol.* As a side contribution we also

perform the first computer-aided verification of a variant of the fundamental BGW protocol for passive and static active adversaries (with computational security in the latter case). In what follows, we outline the main contributions of this work.

Specification and computer-aided proofs of (P)MPC in EasyCrypt. Our proof is performed in the computation model, using the game-based infrastructure provided by EasyCrypt's engine. We make use of the real/ideal paradigm to specify our security notions. We define an environment that is able to select inputs for each party involved in the protocol. The environment has the ability to trigger an adversary that actively corrupts parties (change their inputs, remove them from the execution, etc.) via oracles. This adversary interacts with an *evaluator* that either retrieves to the adversary information from a real execution of the protocol or from an ideal and simulated one. The adversary redirects this information to the environment that tries to distinguish between the two possibilities mentioned. The malicious setting requires a computational bound in our proofs due to the use of the Decision Diffie-Hellman (DDH) assumption in the underlying verifiable secret sharing (VSS).

To accomplish the computer-aided verification we start by identifying appropriate levels of abstraction for MPC protocols (in terms of specification, security and composition) to allow the simplification and reuse of proof steps across the main proof. This abstract structure is an interesting side contribution of this work. It is general enough to be reused since it accommodates many possible instantiations of both MPC and secret sharing protocols. It can be a starting point for other machined-checked MPC proofs to be performed in the future. We also provide a collection of useful lemmas proven in the abstract (such as composition lemmas); this means that any user that wants to leverage advantage of our abstract architecture already has a set of lemmas that can be carried out to concrete instantiations with very little effort and in a mechanical way. Our abstract structure can then be used in other MPC proofs and/or can be tested with different concrete implementations of MPC protocols, other than the ones we provide. We succinctly show how the abstract framework can be reused in Section 3. The proof is completed by providing concrete instantiations that match the abstract definitions, e.g., a variant of the BGW [16] protocol in case of MPC, and in addition to standard and proactive secret sharing, proactive secret sharing for dishonest majorities [11, 30, 33].

Verified extraction tool-chain. In order to obtain a verified implementation of the concrete protocol instantiations defined in EasyCrypt, we developed a new extraction tool-chain for EasyCrypt. We use the Why3 framework [36, 37] as an intermediate layer in the extraction process; this allows us to use Why3's new powerful extraction mechanism [58] to obtain verified implementations of EasyCrypt descriptions in multiple target languages. In this work we only extract executable OCaml code, but our pipeline can be extended to extract C code with some additional work.

As a side, an important feature of our verified MPC evaluator, or the (proactive) secret sharing components thereof, is the possibility of applying it in other contexts and in other projects. For example, it can be composed with a verified arithmetic circuit generator to build a verified MPC stack similarly to that for the two-party case in previous work [4].

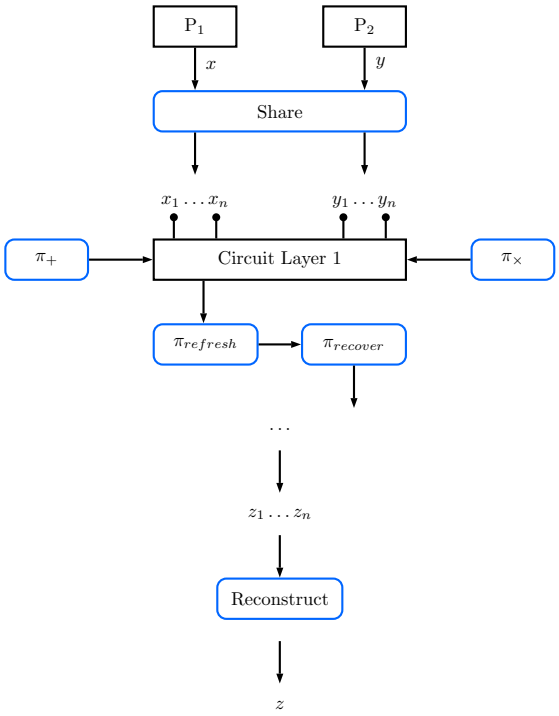

Figure 1: Overview of the verified (P)MPC evaluator; verified sub-protocols are highlighted in blue. Parties first share their private inputs (via the `Share` protocol) which are then passed to the evaluator. The evaluator interprets computation as an arithmetic circuit using verified implementations of addition (π_+) and multiplication (π_x) protocols based on the BGW [16] protocol. Parties locally compute additions and halt on every multiplication, where they synchronize executions. These protocols can be sequentially composed with the `Refresh` ($\pi_{refresh}$) and `Recover` protocols ($\pi_{recover}$) to provide proactive security. At the end of the evaluation, the result of the computation is obtained by reconstructing the output shares via a `Reconstruct` protocol.

A Note on Universal Compostability (UC). Our EasyCrypt formalizations are not in the UC framework. In parallel to our work, Stoughton *et al.* formalized in EasyCrypt a UC proof of security of secure message communication using a one-time pad generated using the Diffie-Hellman key exchange [9]. To the best of our knowledge their work does not cover (P)MPC. Extending our work to the UC framework (possibly using the framework of Stoughton *et al.*) is an interesting avenue for future work but outside the scope of this paper.

[9] https://github.com/easyuc/EasyUC

1.3 Overview of Operation of the Verified (P)MPC Evaluator

The architecture of our evaluator is shown in Figure 1, where verified sub-protocols are highlighted in blue. Computing parties start by sharing their private inputs (via the `Share` protocol), shares are then passed to our evaluator which is running on each party. The evaluator is able to interpret arithmetic circuits using verified implementations of an addition (`add` or π_+) and multiplication (`mul` or π_x) protocols based on a variant of the BGW protocol; in this variant of BGW we use a computationally secure verified secret sharing, VSS, scheme. Parties can locally compute additions and will halt on every multiplication protocol, where they synchronize executions. These protocols can then be combined with the `refresh` (or $\pi_{refresh}$) and `recover` protocols (or $\pi_{recover}$) to achieve proactive security. At the end of the evaluation, the result of the computation can be obtained by reconstructing the output shares of the verified secure evaluator via the `Reconstruct` protocol.

The `Share`, `Reconstruct` and the subsequent protocols for evaluating arithmetic gates are implemented and machine-checked in EasyCrypt. Our specifications and proofs for proactive secret sharing are based on the work by Dolev *et al.* [30], Eldefrawy *et al.* [33] and Baron *et al.* [11] for the dishonest majority setting. When we extend the protocols to MPC, we follow the BGW [16] protocol which only deals with honest majority. One of the important contributions of our work is completing the first computer-aided verification of a variant of the BGW protocol for passive and (static) active adversaries.

Limitations. Formalization of underlying mathematical structures and basic components (i.e., finite fields and cyclic groups and randomness generation), as well as the formalization of a Reed-Solomon decoder (e.g., the Berlekamp-Welch algorithm), is out of scope of this work and is left abstract in our EasyCrypt code. Our Trusted Computing Base (TCB) includes precisely the tools used to instantiate these abstract components: Cryptokit, used to instantiate randomness generation, the OCaml zarith library, used to instantiate finite fields and cyclic groups and finally ocaml-reed-solomon-erasure, an OCaml implementation of a Reed-Solomon decoder.

1.4 Paper Outline

In Section 2, we provide informal descriptions of the cryptographic primitives and protocols that are used in this paper. Section 3 describes how EasyCrypt is used to obtain a concrete proof of security for the aforementioned primitives and protocols. We describe in Section 4 our EasyCrypt extraction tool-chain and how it is used to synthesize a concrete implementation of the evaluator. Performance of the extracted implementations is in Section 5. We finish up by summarizing related work in Section 6, and concluding the paper with a discussion of future research directions in Section 7.

2 PRELIMINARIES: (PROACTIVE) SECRET SHARING AND MPC

Secret sharing. In secret sharing [19, 64], a secret s is shared among n parties such that an adversary corrupting up to t parties cannot learn s, while any $t+1$ parties can recover s. A secret sharing protocol consists of two sub-protocols: `Share` and `Reconstruct`. Initially, secret sharing schemes only considered (exclusively) *passive* or *active adversaries*, and later work [50] generalized this to mixed adversaries.

Proactive secret sharing (PSS). The security of secret sharing should be guaranteed throughout the entire lifetime of the secret. The notion of proactive security was first suggested by Ostrovsky and Yung [57], and applied to secret sharing in [48]. It protects against a mobile adversary that can change the subset of corrupted parties over time. Such an adversary could *eventually gain control of all parties* over a long enough period, but is limited to corrupting less than t parties during the same period. In this work, we use the definition of PSS from [31, 50]: in addition to `Share` and `Reconstruct`, a PSS scheme contains a `Refresh` and a `Recover` sub-protocols. `Refresh` produces new shares of s from an initial set of shares. An adversary that controls a subset of the parties before the refresh and the remaining subset of parties after, will not be able to reconstruct the value of s. `Recover` is required when one of the participant is rebooted to a clean initial state. In this case, the `Recover` protocol is executed by all other parties to provide shares to the rebooted party. Ideally such rebooting is performed sequentially for randomly chosen parties at a predetermined rate – hence the "proactive" security idea. In addition, `Recover` could be executed after an active corruption is detected. While most of the literature on PSS focuses on the honest majority setting [10, 12, 23, 48, 57, 63, 69, 71], the first PSS with dishonest majority was proposed by Dolev et al. in [31]. Standard (linear) secret sharing schemes store the secret in the constant coefficient of a polynomial of degree $< n/2$, an adversary that compromises a majority of the parties would obtain enough shares to reconstruct the polynomial and recover the secret. Instead, [31] leverages the gradual secret sharing scheme of [50] constructed against mixed (passive and active) adversaries, and introduces a PSS scheme robust and secure against $t < n-2$ passive adversaries, or secure but not robust (with identifiable aborts) against either $t < n/2-1$ active adversaries or mixed adversaries (k active corruptions out of $n-k-1$ total corruptions). *As part of our work we implement and verify two versions of PSS. The first PSS is based on the standard Shamir scheme for honest majorities and serves as a foundations for our BGW-based MPC. The second PSS is one for dishonest majority based on gradual secret sharing [31]. However, we stress that we do not have an MPC evaluator for a dishonest majority based on the latter secret sharing scheme.*

Secure Multiparty Computation (MPC). The BGW protocol [16] is one of the first MPC protocols and can be used to evaluate an arithmetic circuit (over a field $\mathbb{F}$) consisting of addition and multiplication gates. The protocol is mainly based on Shamir's secret sharing, where parties share their inputs. Addition can be performed locally, with each party adding its shares bu multiplication requires communication. To evaluate multiplication, parties locally multiply the shares they hold to obtain a $2t$ sharing of the product, then perform a degree reduction by using Lagrange coefficients (this only requires

linear operations) to obtain the sharing of the free term of that polynomial.

Proactive MPC (PMPC). PMPC can be regarded as special case of MPC that remains secure in the proactive security model [57]. In [10], Baron et al. constructed the first (asymptotically) efficient PMPC protocol by using an efficient PSS as a building block. The key idea consists in dividing the protocol execution into a succession of two kinds of phases: *operation phases* (the computation of a circuit's layer with addition and multiplication gates, e.g., as in BGW) and *refresh phases*.

3 MACHINE-CHECKED SECURITY PROOFS FOR SECRET SHARING & MPC

In this section, we first describe what we prove in EasyCrypt, which represents the first step in achieving a verified implementation of an MPC evaluator. Our proof approach consists of two main steps: 1) formalizing in EasyCrypt an abstract framework that captures the desired behavior that we aim for our evaluator; and 2) an instantiation step that provides concrete realizations (also in EasyCrypt) of the protocols and primitives defined in the abstract setting. The intuition is that we will use the abstract framework to perform proofs at a high and modular level, and then propagate those results downwards to concrete implementations with little (proof) overhead. In fact, our abstract framework is modular enough to be reused and applied to other MPC instantiations. We provide a concrete example of this later (see subsection 3.4).

We start the presentation of our EasyCrypt implementation with the description of the abstract framework ($\sim$ 2K LoC), that can be subdivided into two independent modules, one for secret sharing and one for MPC protocols. We want to establish a relation between both in order to reduce the security of the overall evaluator to the security of an arbitrary secret sharing scheme and to the security of an arbitrary PMPC protocol, as shown in Theorem 3.1. We end the section with an explanation of the instantiation step, accounting for 7K LoC, of which 1K comprises protocol specifications.

3.1 Proof overview

We first give an overview of the proving process, giving a high-level outline of it. As already mentioned, we first modeled the desired behaviour of our evaluator by means of an abstract and modular setting, encompassing the abstract framework to specify abstract functional definitions and security properties for all cryptographic constructions (primitives, algorithms, and protocols) that are used in this work. Since it does not take into account any functional specification of the components involved, it makes sense to use it to establish as much results as possible. Concretely, many results (such as composition theorems) can be obtained by reasoning at an abstract level, which can then be easily carried out to concrete instantiations. For example, one can prove that two primitives (that match some security definitions) can be composed to yield another primitive with a powerful security definition. The construction of a CCA-secure encryption scheme via a CPA-secure encryption scheme and an UF-secure MAC scheme, without specifying any of the two, is a good example of this approach [52].

Our abstract framework is comprised of two abstract modules: one for secret sharing and another for MPC protocols. The first one defines an abstract construction of a secret sharing scheme and specifies three security definitions for it: passive (honest-but-curious), *integrity*, and malicious (verifiable) secret sharing. At this level, we prove that security of a verifiable secret sharing scheme can be reduced to security of a passive secret sharing scheme that ensures integrity of the shares (i.e., any modification of the shares is detectable by the parties involved in the protocol). We also prove that share integrity can be ensured if a commitment scheme is used along side a secret sharing scheme, thus proving that an honest-but-curious secret sharing scheme can be composed with a commitment scheme in order to build a verifiable secret sharing scheme.

The second one provides an abstract formulation of MPC protocols and four security definitions: passive, malicious, *random*, and *proactive*. The two latter security definitions are enough to accommodate the desired behavior of the refresh and recover protocols of the proactive model, respectively. We define a series of (sequential) composition lemmas involving the security notions above, from which we highlight three:

(1) malicious $\circ$ malicious $\Rightarrow$ *malicious*
(2) *random* $\circ$ malicious $\Rightarrow$ *random*
(3) *proactive* $\circ$ *random* $\Rightarrow$*proactive*.

Armed with these three lemmas, we can state that one is able to achieve a *proactive* secure MPC protocol by combining smaller *malicious* MPC protocols with a *random* and a *proactive* protocol. Concretely to our case, we want to have a sequence of add and mul protocols (that perform the evaluation of the desired arithmetic circuit) ending with a refresh and a recover protocol to wrap the security of the execution.

Finally, we use these two abstract constructions to specify Theorem 3.1 that upper bounds the (proactive) security of the evaluator with the security of the cryptographic constructions it encompasses without considering concrete realizations.

THEOREM 3.1. *For all* ProAct *adversaries $\mathcal{A}$ against the EasyCrypt implementation* Evaluator*, there exist efficient simulator* S *and adversaries $\mathcal{B}_{\mathsf{VSS}}$ and $B_{\mathsf{Proactive-MalSS}}$, such that:*

$$\mathsf{Adv}^{\mathsf{Proactive}}_{\mathsf{Evaluator},\mathsf{S}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathsf{VSS}}(\mathcal{B}_{\mathsf{VSS}}) + \mathsf{Adv}^{\mathsf{Proactive-MPC}}(\mathcal{B}_{\mathsf{Proactive-MPC}})$$

where $\mathsf{Adv}^{\mathsf{VSS}}(\mathcal{B}_{\mathsf{VSS}})$ *and* $\mathsf{Adv}^{\mathsf{Proactive-MPC}}(\mathcal{B}_{\mathsf{Proactive-MPC}})$ *represent the advantages against the (verifiable) secret sharing scheme and against the proactive MPC protocol to be executed.*

Our EasyCrypt proof ends with the instantiation step, where concrete realizations of a secret sharing scheme, commitment scheme, and MPC sub-protocols are specified and tied to the abstract definitions. The main challenge in the instantiation step is to prove that the concrete definitions are actually valid instantiations of the desired primitives, since all results obtained in the abstract framework (namely Theorem 3.1), are easily carried out to the concrete constructions once they are proven to be valid instantiations of primitives.

An important component of this work was the development of a verified EasyCrypt polynomial library, that provides a series of types and operators that deal with polynomials defined over a finite field. We include a description of such library in Appendix A.

In what remains of this section, we will detail our EasyCrypt implementation, explaining the intermediate steps that were followed in order to derive Theorem 3.1, namely how to build the two cryptographic primitives that define the security of the evaluator. We will also provide EasyCrypt code snippets for relevant components of this work. Due to space constrains, we will not be able to provide an extensive explanation of the proof, as we will focus on just concrete instantiations of secret sharing schemes or MPC protocols. We refer the reader to the full version of this paper [34] for a more detailed and complete description of our EasyCrypt deployment.

3.2 Secret Sharing in EasyCrypt

Secret sharing, and its verifiable and proactive variations, plays a central role in our (proactive) secure evaluator. It is the first primitive to be executed and any security violation in this component can compromise the entire security of the evaluator. Different shares of different secrets must be indistinguishable and should also be non-corruptible. Our first goal is to formally represent a verifiable secret sharing scheme and to derive a concrete EasyCrypt implementation of it that could be extracted to executable code via our extraction tool-chain.

We first describe our abstract secret sharing framework, pointing out how it can simplify the proofs that could be much more complex if they were to be carried out considering concrete realizations of the involved primitives. We then show how the abstract secret sharing primitives can be instantiated, and how the results proven in the abstract setting are naturally propagated (down) to concrete instantiations of these schemes.

Abstract Secret Sharing. The structure of the abstract secret sharing framework is shown in Figure 2. In this figure, we depict abstract cryptographic primitives as rounded rectangles and security definitions as red rectangles. If some security notion applies to some primitive, then the two are connected by a solid arrow. The same way, if a primitive is used as an abstract building block in the security of another primitive, it will be enclosed inside its rounded rectangle as a green circle. Security proofs are represented as dashed arrows. Labeled arrows represent proofs that are propagated to the concrete instantiation of the abstract modular framework.

The main result shown in Figure 2 is the definition of a verifiable secret sharing scheme as the composition of an honest-but-curious secret sharing scheme with an unforgeable commitment scheme. Our formalization of such results starts with the abstract definition of a secret sharing scheme structure, illustrated in Figure 3. A secret sharing scheme is parameterized by 4 integers: i. n - number of parties involved in the protocol; ii. k - number of secrets to be shared; iii. d - number of parties needed to reconstruct the secret; and iv. t - corruption threshold. The addition of the k parameter may be an exaggeration for most secret sharing protocols but it allows the specification of *batch* secret sharing schemes [40].

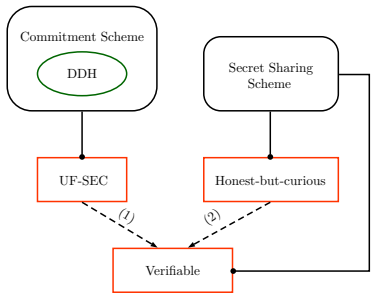

Figure 2: Secret sharing abstract framework. We define two cryptographic primitives and provide three security definitions, one for commitment schemes and two for secret sharing schemes. The same secret sharing abstraction can be used for both security definitions.

Next, the interface defines both the types and operations involved in the sharing protocol, including party identifiers, secrets, shares and randomness types and, finally, the actual share and reconstruct operators. We highlight the inclusion of *p_id_set*, which will contain all party identifiers. This parameter will be very useful when proving correctness properties of both secret sharing and subsequent MPC protocols.

```
theory SecretSharingScheme.
  op n : int. (* Number of parties *)
  op k : int. (* Number of secrets *)
  op d : int. (* Number of parties needed to reconstruct *)
  op t : int. (* Corruption threshold *)
  type p_id_t. (* Party identifier *)
  op p_id_set : p_id_t list. (* Set of parties involved *)
  type secret_t. (* Secret type *)
  type share_t. (* Share type *)
  type shares_t = (p_id_t * share_t) list. (* Set of shares *)
  op dshare : share_t distr. (* Share distribution *)
  type rand_t. (* Randomness *)
  (* Share operation *)
  op share : rand_t -> secret_t -> shares_t.
  (* Reconstruct operation *)
  op reconstruct : shares_t -> secret_t option.
end SecretSharingScheme.
```

Figure 3: Abstract secret sharing scheme

Randomness is modeled as an explicit operator parameter. This allows us to write probabilistic operations with deterministic flavour. For example, we define the share operator as **op** share : rand_t → secret_t → shares_t, which is a deterministic operator. Nevertheless, the sharing procedure of a secret sharing scheme is, naturally, a probabilistic algorithm. Lifting it to a probabilistic operator would involve writing it as **op** share : secret_t → shares_t distr. Semantically, it means that the operator would be outputting probability distribution on shares and that one would need to sample from this distribution in order to get a valid set of shares. This lift would have very little effect on the proof but would, however, impact the code extraction process as it would be infeasible to extract such probabilistic operators.

We define three security requirements for secret sharing: *honest-but-curious*, *integrity* and *verifiable*. For a full description of these security definitions, we refer the reader to [34] for their details.

Abstract commitment scheme. To guarantee integrity of generated shares, a dealer needs to commit to them. A commitment scheme can be defined based on the type of randomness used, the type of messages and the type of the commits it produces. Figure 4 shows the abstraction of a commitment scheme. Interestingly, the same abstraction could also be used to define other integrity mechanisms, e.g., message authentication codes.

```
theory CommitmentScheme.
  type rand_t. (* Randomness *)
  type msg_t. (* Message type *)
  type commit_t. (* Commit type *)
  (* Mac operation *)
  op commit : rand_t -> msg_t -> commit_t.
  (* Verify operation *)
  op verify : msg_t -> commit_t -> bool.
end CommitmentScheme.
```

Figure 4: Abstract commitment scheme

Nonetheless, such abstraction is not enough for the construction of a verifiable secret sharing scheme. In fact, the way the commitment scheme is written, it only works for a single share, instead of multiple shares as desired in our composition proof. We thus define a list variation of a commitment scheme that applies the commitment scheme to a list of messages in order to smooth the composition with an honest-but-curious secret sharing scheme. With this purpose, we defined a new theory ListCommitmentScheme (Figure 5) which is parameterized by any commitment scheme. This means that, in order to come up with a list variation of a given commitment scheme, one only needs to plug the *single* version of the commitment scheme to theory ListCommitmentScheme. The modularity of the secret sharing abstract framework can also be verified here, as the described list version will work for any commitment scheme that is plugged to it.

```
theory ListCommitmentScheme.
  (* Abstract commitment scheme *)
  clone import CommitmentScheme as CS.
  clone import CommitmentScheme as ListCS with
    type rand_t = CS.rand_t,
    type msg_t = CS.msg_t list,
    type commit_t = CS.commit_t,
    op commit (r : rand_t) (m : msg_t) =
      map (CS.commit r) m,
    op verify (m : msg_t) (c : commit_t) =
      all ((=) true) (map2 CS.verify m c).
end ListCommitmentScheme.
```

Figure 5: Abstract *list* commitment scheme

We specify an unforgeability security definition for commitment schemes that can be consulted in [34].

```
theory AbstractVerifiableSecretSharing.
  ...
  clone import SecretSharingScheme as AVSS with
    ...
    type share_t = SS.share_t * CS.commit_t,
    type rand_t = SS.rand_t * CS.rand_t,
    op share (r : rand_t) (s : secret_t) =
      let (rs,rc) = r in
      let ss = SS.share rs s in
      let cs = LCS.commit rc ss in
      merge ss cs,
    op reconstruct (css : shares_t) =
      let ss = unzip1_assoc css in
      let cs = unzip2_assoc css in
      if LCS.verify ss cs then SS.reconstruct ss else None.
  ...
end AbstractVerifiableSecretSharing.
```

Figure 6: Abstract verifiable secret sharing scheme in EasyCrypt.

Abstract verifiable secret sharing. We conclude our description of the abstract secret sharing framework by describing our modular composition proof of a semi-honest secret sharing scheme with an unforgeable commitment scheme, yielding a verifiable secret sharing scheme. We do not claim any novelty here, this is a standard technique in previous verifiable secret sharing articles [1, 7, 24, 25, 35, 38, 41, 42, 49, 51, 53, 59–61].

The primitives can be composed as shown in Figure 6. Our composition proof proceeds similar to the *encrypt-then-MAC* for CCA-secure encryption schemes. We leave the passive secret sharing scheme abstract but we force some types of the commitment scheme, namely that it produces commits to associations between a party identifier and its respective share. Thereby, the list variation of this commitment scheme would be producing commits to a map between party identifiers and shares, which is precisely the type of values outputted by algorithm share presented in Figure 3.

The composition of the two primitives is a new secret sharing scheme where shares now carry a commitment with them. Protocols share and reconstruct are specified in the expected way: i. share first shares the secret according to the honest-but-curious secret sharing scheme, before producing commits to the generated shares; and ii. reconstruct first verifies the integrity of the shares (if there was some malicious modifications). If no integrity breach was found, it proceeds with the reconstruction of the secret according to the passive secret sharing scheme. If it was unable to attest the validity of some share, the algorithm fails and produces no output.

Theory AbstractVerifiableSecretSharing also includes a series of security instantiations. The goal is to prove that malicious security for AVSS can be obtained as a combination of other security guarantees, namely the unforgeability of the commitment scheme and the indistinguishability of the passive secret sharing scheme.

The security proof of the verifiable secret sharing construction in Figure 6 was done in two steps. We start by showing that malicious security of AVSS can be reduced to its passive security and its integrity. Finally, we individually show that passive security of the verifiable secret sharing scheme can be reduced to the passive security of the underlying secret

sharing scheme and that share integrity can be reduced to the unforgeability of the underlying commitment scheme.

Concrete secret sharing. We now show how to make use of the modular abstract framework to produce concrete instantiations of cryptographic primitives that are compatible with our EasyCrypt extraction tool, focusing on the achievement of a verifiable secret sharing scheme. Based on the afore described modular proof, one only needs to provide a valid realization of an honest-but-curious secret sharing scheme and of an unforgeable commitment scheme. The verifiable secret sharing scheme is derived easily via specifications in Figure 6, as presented in Figure 7. In the figure, if some arrow is labeled, it means that the result is already proven in the abstract framework and that it can be reused here (hence it does not represent any significant verification overhead). The proofs that were made in this phase (proofs that concrete implementation of a primitive achieves the desired security) are represented as solid arrows.

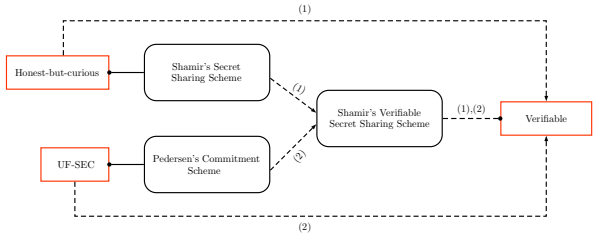

Figure 7: Concrete instantiation of the secret sharing framework. Concrete Shamir's verifiable secret sharing scheme is easily obtained via a concrete instantiation of Shamir secret sharing scheme and of Pedersen's commitment scheme. The major challenges in this step are the security proofs of Shamir secret sharing schem and of Pedersen's commitment scheme.

For our proactive secure evaluator, we choose to implement the verifiable version of Shamir's secret sharing scheme [65] combined with Pedersen's commitment scheme [60]. The EasyCrypt implementation of Shamir's secret sharing scheme is detailed in Figure 8, where *F.t* represents the type of elements of a finite field.

```
theory ShamirSecretSharingScheme.
  ...
  clone import SecretSharingScheme as Shamir with
    type secret_t = F.t,
    type share_t = F.t,
    type rand_t = polynomial,
    op share (r : rand_t) (s : secret_t) =
      map (fun x => (x,eval x r)) p_id_set.
    op reconstruct (ss : (p_id_t * share_t) list) =
      Some (interpolate CyclicGroup.FD.F.zero ss).
  ...
end ShamirSecretSharingScheme.
```

Figure 8: Shamir secret sharing scheme

The next step was to specify a concrete commitment scheme. Because we were interested in exploring the homomorphic properties of commitments, we chose to implement Pedersen's commitment scheme, reproduced in Figure 9.

```
theory PedersenCommitmentScheme.
  ...
  op multiplicator (l : group list) : group =
    foldr (CyclicGroup.( * )) g1 l.
  clone import CommitmentScheme as PedersenCS with
    type rand_t = polynomial * polynomial,
    type msg_t = t * t,
    type commit_t = t * ((int * group) list),
    op commit (r : rand_t) (m : msg_t) =
      let (rs,rc) = r in let (m,sh) = m in
      let rz = zip rs rc in
      (eval m r, map (fun x => ((fst x).`expo, g^(fst x).`coef *
            h^(snd x).`coef)) rz),
    op verify (m : msg_t) (c : commit_t) : bool =
      let (c, gsh) = c in
      let (m,sh) = m in
      g^sh*h^c = multiplicator (map (fun x => snd x ^ (m ^ fst x
           )) gsh).
  ...
end PedersenCommitmentScheme.
```

Figure 9: Pedersen commitment scheme

Finally, we can instantiate the abstract verifiable secret sharing construction using the concrete passively secure secret sharing scheme (ShamirSS, inside theory ShamirSecretSharingScheme) and the concrete unforgeable commitment scheme (PedersenCS, inside theory PedersenCommitmentScheme). This instantiation step is actually very simple and can be done with very little overhead. An EasyCrypt implementation of the verifiable secret sharing version of Shamir's secret sharing scheme can then be obtained simply by plugging ShamirSS and PedersenCS to the abstract verifiable secret sharing construction.

PSS with dishonest majority. We also implemented a proactive (gradual) secret sharing scheme [30, 33] which is secure against a dishonest majority of mixed adversaries - detailed in Section 4 - where we use it for an example-driven presentation of how the EasyCrypt extraction tool works. Formalization and extracted executables of this scheme can be of independent interest beside the BGW-based MPC part.

3.3 MPC Protocols in EasyCrypt

We divide the MPC sub-protocols we specified into two main categories: (1) *arithmetic protocols*, used to perform actual circuit computation; these are the addition (add) and multiplication (mul) protocols; and (2) *"security" protocols*, used to ensure proactive security throughout the evaluation of the circuit; protocols refresh and recover will be used in this context.

The circuit to be executed will be composed of a series of addition and multiplication protocols, interpolating with refresh and recover in order to maintain the evaluation secure. Informally, we want protocols add and mul to achieve *privacy*, meaning that we will only want for the communication traces and output shares of these protocols to leak no information about the input shares involved in the protocol. Afterwards, an execution of refresh would re-randomize the outputs. We call such security notion *random*. Lastly, recover can be used in order to prevent an adversary from potentially

corrupting all nodes of the system. We say protocol recover achieves *proactive* security. All the afore mentioned security notions consider malicious adversaries. This high-level description is summarized in Figure 10, where we depict our abstract modular framework for MPC protocols. This architecture leverages the security definitions found in other simular tools such as Sharemind [20, 21] or [5]. Nevertheless, these two works only consider passive security, while our evaluator provides proactive security.

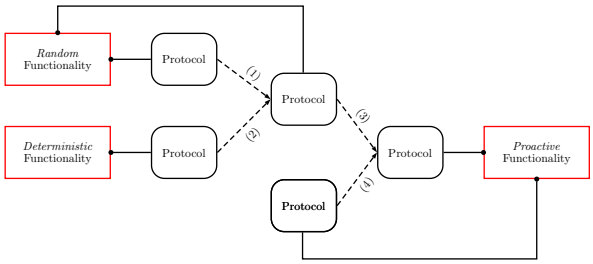

Figure 10: MPC abstract framework. Protocols can realize three functionalities - *deterministic, random* and *proactive. Smaller* protocols can also be composed in order to achieve stronger security guarantees.

Abstract MPC. At the base of our MPC abstract framework lies the abstraction of an MPC protocol (Figure 11). The abstraction defines an operator *prot* which models the functional behaviour of the protocol, with parties entering the protocol with their inputs and randomness information and ending with some output. In a possible instantiation, one could specify individual party operations that can then be called inside *prot* operator, which would model communication.

```
theory AbstractProtocol.
  type p_id_t. (* Party identifier *)
  op p_id_set : p_id_t list. (* Set of parties involved *)
  type input_t. (* Party input *)
  type inputs_t = (p_id_t * input_t option) list. (* Set of
       party inputs *)
  type output_t. (* Party output *)
  type outputs_t = (p_id_t * output_t option) list. (* Set of
       party outputs *)
  type rand_t. (* Party randomness *)
  type rands_t = (p_id_t * rand_t option) list. (* Set of party
        randomness *)
  type conv_t. (* Party "conversation" *)
  type convs_t = (p_id_t * conv_t option) list. (* Set of party
        "conversations" *)
  (* Executes the protocol *)
  op prot : rands_t -> inputs_t -> (convs_t * outputs_t).
end AbstractProtocol.
```

Figure 11: Abstract MPC protocol

The proposed abstraction not only fits complete instantiations of protocols but also allows one to *split* the same protocol into multiple *smaller* protocols and then compose them into a *bigger* protocol. For example, the refresh protocol can be written as a composition of a protocol where every party shares the value 0 (zero), a protocol where every party sends to each other the corresponding shares and a protocol where every party adds the received shares to the current share. This approach is actually ideal when reasoning about the security of a protocol against malicious adversaries. Intuitively, one can prove the desirable level of security for the individual protocols and, subsequently, derive the security of the composed protocol using the composition lemmas that we describe with more detail later in this section. Note that this approach contemplates possible misbehaviors in the middle of the protocol execution, which is a desirable property for malicious secure protocols and, probably, the most challenging aspect to model.

In our evaluator, MPC protocols may realize three types of functionalities. Similarly to the protocol abstraction, we define functionalities that will execute in one block and functionalities that will execute by phases. In the *private* functionality, protocols must realize some arithmetic operation. Intuitively, this functionality receives unshared inputs and computes the arithmetic operation over raw inputs instead of over shared values. Regarding *random* functionality, protocola must re-randomize the input shares. In short, executing the protocol must output a result indistinguishable from freshly sharing a secret after reconstructing it. Finally, in the *proactive* functionality, protocols must recover a corrupted party to a non-corrupted state. Informally, recovering parties should receive new shares that would invalidate the previous ones. EasyCrypt snippets for the three functionalities can be found in the extended version of this paper.

These functionalities are tied to a specific *Real* $\sim$ *Ideal* security experiences. For *private* and *random* security, our security experiencies are extensions to the malicious setting of the ones defined in [5, 20], while for *proactive* security we use the ones formally defined in [33]. Likewise, we define three different types of simulators. This is due to the fact that simulators will have different goals, according to the security asset.

We modeled the security definitions by means of EasyCrypt modules in the following way. An *environment* Z will try to distinguish between a real execution of the protocol or the simulated one. In order to do so, Z can ask an *adversary* A to execute either the protocol or to simulate it (behaviour defined by some bit *b*). Since we are interested in malicious adversaries, this adversary will be grant access to oracles (that are to be executed at the end of every protocol stage):

- *corrupt* - getting access to party
- *corruptInput* - modify some corrupted party phase input
- *abort* - remove some party from the protocol

At the end, A will provide Z with information (depending on the security defintion) that Z will use to make its guess. We refer the reader to [34] for a complete view of the representation of the security definitions.

The functionalities, security experiences and simulators described above define security for individual protocols but not for the entire evaluator. Intuitively, we want the overall evaluator to be a composition of smaller protocols for which it is simpler to prove security. Subsequently, we want to derive security for the entire system by relying on composition lemmas surrounding the MPC protocols. Informally, we want to

evaluate a sequence of addition and multiplication protocols, keeping the evaluation *private*. The circuit will then reach a refresh or a recover protocol, and will assume the security each protocol provides. We thus need three composition lemmas:

(1) The composition of two *private* protocol yields a *private* protocol.
(2) The composition of a *private* protocol with a *random* protocol yields a *random* protocol.
(3) The composition of a *random* protocol with a *proactive* protocol yields a *proactive* protocol.

To finish this subsection, we highlight how EasyCrypt can be used in order to define the composition of a *private* and a *random* protocols and how to specify its security.

Figure 12 details the specification of the composed protocol $\Pi = \pi_2 \circ \pi_1$. The structure is simple. Each party i involved in protocol Π will input two shares x_i and y_i of the values x and y, respectively, and will end the execution of Π with one share z_i of some value z, that will be the randomized result of the arithmetic operation realized by protocol π_1. Inputs x_i and y_i are also the inputs of protocol π_1, whereas the the input of π_2 will be the output of π_1.

The security of the composed protocol can now be reduced to the security of the two protocols. The interesting aspect about this proof is that it can be performed in an abstract and modular way. Consequently, in future instantiations, the only concern will be proving security for the small components of a bigger protocol, since the composition lemmas can be applied. The reason why the proof can be modular is because the composed simulator and functionalities do not need to be restricted to any particular behavior or description but can simply be defined as the sequential execution of S_1 and S_2 and F_1 and F_2. Figure 12 demonstrates the security set up for the composed protocol Π.

The intuition behind the security proof is that every real execution of a protocol can be replaced by the ideal one. At the end, instead of executing the two protocols sequentially, the security experience will be executing the two simulators sequentially, therefore connecting the real and ideal worlds.

Concrete MPC Instantiation. We implemented in EasyCrypt protocols add, refresh and recover specified in [33]. Protocols add and refresh do not introduce a significant implementation overhead since add is a simple local protocol and refresh can be seen as a composition of share (of a secret equal to 0) and add. Protocol recover is slightly more complex than the previous two. It involves non-recovering parties to sub-share their shares and recovering parties performing polynomial interpolation in order to obtain the new shares. For multiplication, we choose to implement the simplified malicious version of the BGW multiplication protocol [15] described by Asharov and Lindell in [6]. This version is similar to the passive version, with the introduction of a an error correction step that can correct possible subshare corruptions. Figure 13 compiles the instantiation step of MPC protocols.

We now present the EasyCrypt formalization of the recover protocol. Protocols add, refresh and mul can be found in [34]. The EasyCrypt code of the MPC protocols is written party-wise, meaning that we have operators for every stage of the protocol that are then merged in the prot operator. For example, operator nr_pstage1, nr_pstage2 and r_pstage1 in Figure 14 represent the first stage of non-recovering parties, the second stage of non-recovering parties and the first and only stage of

```
theory RandomAfterPrivateComposition.
  ...
  clone import AbstractProtocol as P1.
   clone import PrivateProtocolSecurity as P1PrivSec with
    theory AbstractProtocol <- P1.
  clone import AbstractProtocol as P2 with
    type input_t = P1.output_t.
  clone import RandomFunctionality as RandomFunc with
    type input_t = P1PrivSec.F.output_t.
  clone import RandomProtocolSecurity as P2RandSec with
    theory AbstractProtocol <- P2,
    theory F <- RandomFunc.
  clone import AbstractProtocol as P with
    type input_t = P1.input_t,
    type output_t = P2.output_t,
    type rand_t = P1.rand_t * P2.rand_t.
  clone import RandomProtocolSecurity as PRandSec with
    type F.input_t = P1PrivSec.F.input_t,
    type F.output_t = P2RandSec.F.output_t,
    type F.rand_t = P2RandSec.F.rand_t,
    type F.leak_t = P1PrivSec.leak_t * P2RandSec.leak_t,
    op F.func (r : F.rand_t) (xx : F.input_t) =
      let (y, l1) = P1PrivSec.F.func xx in
      let (z, l2) = P2RandSec.F.func r y in (z, (l1, l2)),
    theory AbstractProtocol <- P.
  ...
  module Simulator(S1 : P1PrivSec.Sim_t) (S2 : P2RandSec.Sim_t)
        ={
    proc simm(l : leak_t, xi : input_t, zi : output_t) = {
      ...
      (l1, l2) <- l;
      (yi, v1) <@ S1.simm(l1, xi);
      v2 <@ S2.simm(l2, yi, zi);
      return ((v1,v2));
    }
  }
  ...
end RandomAfterPrivateComposition.
```

Figure 12: Composition of a *private* protocol with a *random* protocol

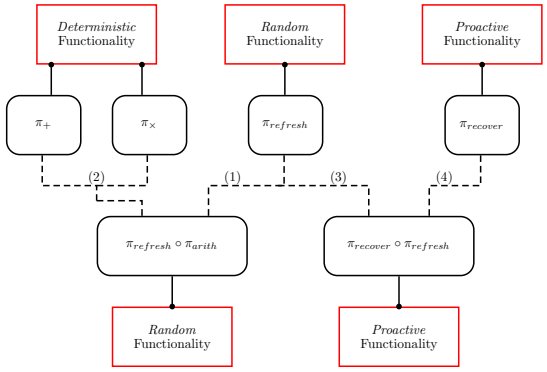

Figure 13: Concrete instantiation of MPC framework. π_+ and $\pi_\times$ are used to evaluate an arithmetic, while $\pi_{refresh}$ and $\pi_{reconstruct}$ are used to ensure security of the evaluation. Security of the composed protocols is easily derive from the abstract framework once security is proven for concrete instantiations.

recovering parties in the recover protocol, respectively. The outputs of those stages are the party state (values that need to be carried out throughout the execution of the party) and some message, which can either be a broadcast (like bdcst1_t in nr_pstage1) or a simple message (like msg2_t in nr_pstage2). The prot operator is used to depict the global execution of the protocol, including simulation of party communication.

```
theory RecoverProtocol.
  ...
  clone import Protocol as Recover with
    type pstate1_t = shares_t,
    op nrpstage1 (r : rand_t) : pstate1_t =
      with r = RRP _ => []
      with r = RNRP rs => share rs F.zero,
    op nrpstage2 pid i ss =
      foldr (fun (x : share_t) (acc : share_t) =>
            if verify (pid, fst x) (snd x) then
              AdditionProtocol.pexec (x,acc)
            else acc) i ss,
    op rparty pid r ss =
      with r = RRP rc =>
        let o = interpolate pid (map (fun pidsh => (fst pidsh,
             fst (snd pidsh))) ss) in
        let c = commit rc (pid,o) in (o,c)
      with r = RNRP _ => (witness, witness),
    op prot rs iss =
      let rp = fst (snd (head witness iss)) in
      let nrp = rem rp p_id_set in
      let iss = map (fun pidi => let (pid,i) = pidi in (pid, snd
           i)) iss in
      let pst1 = map (fun pid => (pid, nrpstage1 (oget (assoc rs
           pid)))) nrp in
      let cs = map (fun pid => (pid, map (fun ss => snd ss) (
           oget (assoc pst1 pid)))) nrp in
      let sss = map (fun pid => (pid, map (fun idss => oget (
           assoc (snd idss) pid)) pst1)) nrp in
      let os = map (fun pid => (pid, nrpstage2 pid (oget (assoc
           iss pid)) (oget (assoc sss pid)))) nrp in
      let cs = map (fun pid => (pid, oget (assoc cs pid) ++ (
           oget (assoc sss pid)))) nrp in
      let orp = rparty rp (oget (assoc rs rp)) os in
      let crp = map (fun x => snd x) os in
      Some ((rp, crp) :: cs, (rp, orp) :: os).
end RecoverProtocol.
```

Figure 14: Recover protocol

Following the BGW protocol description from [6], we built a Reed-Solomon code specification in EasyCrypt, that we use to correct possible errors in the shares. We keep this definition abstract for now, and leave its concrete realization for future work.

A simple circuit evaluation function can now be defined. This function, taking a circuit description as input (a series of additions and multiplications), outputs a protocol version of that circuit mapping every addition into an add protocol and every multiplication into a mul protocol. Nevertheless, the protocol description of the circuit should also take care of the introduction of periodic calls to refresh and recover protocols, in order to ensure that the system achieves the desired level of security. We opt to introduce a refresh and a recover protocol at the end of every circuit layer. This decision introduces a performance penalty that could be mitigated if recover and refresh would only be introduced in strategic points of the circuit.

3.4 Reusability of the abstract framework

One of the most interesting features of our work is the propose abstract framework. We claim it is general enough to accommodate multiple possible instantiations of both secret sharing schemes and MPC protocols. The previously shown instantiations are just an example of how it can be used. In this section, we demonstrate that our framework is even able to abstract existing MPC platforms such as Sharemind [20, 21].

Sharemind incorporates a 3-out-of-3 additive secret sharing scheme, which shares elements of a finite field and produces shares which are also elements of the same finite field. The sharing and reconstructing processes are conceptually easy. Sharing outputs three shares, where two are randomly generated and the other one is obtaining by subtracting the secret by the two random shares. The secret can be easily reconstructed by adding all the shares. We show how to instantiate this scheme using our secret sharing abstraction in Figure 15.

```
theory AdditiveSecretSharingScheme.
  clone import SecretSharingScheme with
    op n = 3,
    op k = 1,
    op d = n,
    op t = n - 1,
    type p_id_t = t,
    op p_id_set = [ofint 0, ofint 1, ofint 2],
    type secret_t = t,
    type share_t = t,
    type rand_t = t * t,
    op share (r : rand_t) (s : secret_t) : (int * share_t) list
         =
      let (r1,r2) = r in
      zip p_id_set [r1; r2; s - r1 - r2],
    op reconstruct (ss : (int * share_t) list) : (secret_t)
        option =
      Some (summation (unzip2 ss)).
end AdditiveSecretSharingScheme.
```

Figure 15: Sharemind's additive secret sharing scheme

Similarly to our addition protocol, Sharemind addition (depicted in Figure 16) can be locally computed by simply adding shares.

```
theory SharemindAddition.
  clone import Protocol as SAddition with
    type p_id_t = p_id_t,
    op p_id_set = p_id_set,
    type input_t = share_t * share_t,
    type output_t = share_t,
    type rand_t = unit,
    type conv_t = unit,
    op prot (r : (p_id_t * rand_t) list) (is : (p_id_t * input_t
        )) =
      (map (fun pid => (pid, None)) p_id_set,
      map (fun pid => let x = oget is.[pid] in (pid, fst x + snd
           x)) p_id_set).
end AdditiveSecretSharingScheme.
```

Figure 16: Sharemind's addition protocol

Refreshing shares inside the Sharemind platform involves parties generating a random value and sending that value to the *next* party, i.e. party i sends its random value to party

$i + 1$. Shares are randomized by adding the party's random value and subtracting the received random value. Sharemind's refresh protocol can be found in Figure 17.

```
theory SharemindRefresh.
  clone import Protocol as SRefresh with
    type p_id_t = p_id_t,
    op p_id_set = p_id_set,
    type input_t = share_t,
    type output_t = share_t,
    type rand_t = t,
    type conv_t = t * t,
    op prot (r : (p_id_t * rand_t) list) (is : (p_id_t * input_t
         )) =
      (map (fun pid => let ri = oget r.[pid] in
                       let ri1 = oget r.[pid - 1] in
                       (pid, (ri, ri1))) p_id_set,
       map (fun pid => let ri = oget r.[pid] in
                       let ri1 = oget r.[pid - 1] in
                       let i = oget is.[pid] in
                       (pid, i + ri - ri1)) p_id_set).
end SharemindRefresh.
```

Figure 17: Sharemind's refresh protocol

Finally, the multiplication protocol is slightly more complex than the previous two but is conceptually similar to the refresh protocol as parties will also send information to the party with the next identifier. It involves two calls to the refresh protocol and every party will send the new randomized shares to the *next* party. At the end, the multiplication of two values $\bar{x} = x_1 + x_2 + x_3$ and $\bar{y} = y_1 + y_2 + y_3$ can be obtained by performing $\sum_{i=1}^{3} \sum_{j=1}^{3} x_i \cdot y_j$. We show the EasyCrypt instantiation of Sharemind's multiplication protocol in Figure 18.

We demonstrated that all Sharemind components are actually concrete instantiations of our abstract framework, thus providing evidence of its modularity and generality. All this elements can then be composed in order to obtain the full Sharemind evaluation system.

4 EASYCRYPT EXTRACTION TOOL-CHAIN

Our verified implementation of a (proactive) MPC evaluator is obtained via a new extraction tool-chain for EasyCrypt. The general execution pipeline of the tool-chain is shown in Figure 19. Briefly, an EasyCrypt description is first translated into a WhyML program, which can then be fed to the Why3 [36, 37] platform in order to perform extraction to OCaml using the new Why3 (verified) extraction mechanism [58]. Note that besides making use of Why3 code generation capabilities, one could also use Why3's proving system. For example, one could take the generated WhyML program, annotate it with the desired predicates and use Why3 to discharge the generated verification conditions; for example, one can use Why3 to prove safety about some EasyCrypt code in an automated way. This subject is outside the scope of this work and is an interesting future research direction.

We chose Why3 as an intermediate tool for two reasons. First, the specification languages of both frameworks (EasyCrypt and Why3) are very similar, which simplifies the translation

```
theory SharemindMultiplication.
  clone import Protocol as SMultiplication with
    type p_id_t = p_id_t,
    op p_id_set = p_id_set,
    type input_t = share_t * share_t,
    type output_t = share_t,
    type rand_t = t * t,
    type conv_t = t * t * t * t,
    op prot (r : (p_id_t * rand_t) list) (is : (p_id_t * input_t
         )) =
      let xx = map (fun pid => (pid, fst (oget is.[pid])))
           p_id_set in
      let yy = map (fun pid => (pid, snd (oget is.[pid])))
           p_id_set in
      let r1 = map (fun pid => (pid, (oget r.[pid]).`1))
           p_id_set in
      let r2 = map (fun pid => (pid, (oget r.[pid]).`2))
           p_id_set in

      let (crxx, rxx) = SRefresh.prot r1 xx in
      let (cryy, ryy) = SRefresh.prot r2 yy in

      (map (fun pid => let xi = oget rxx.[pid] in
                       let yi = oget ryy.[pid] in
                       let xi1 = oget rxx.[pid - 1] in
                       let yi1 = oget ryy.[pid - 1] in
                       (pid, (xi, yi, yi1, xi1)) p_id_set,
       map (fun pid => let xi = oget rxx.[pid] in
                       let yi = oget ryy.[pid] in
                       let xi1 = oget rxx.[pid - 1] in
                       let yi1 = oget ryy.[pid - 1] in
                       (pid, xi * yi + xi * yi1 + xi1 * yi))
                            p_id_set).
end SharemindMultiplication.
```

Figure 18: Sharemind's multiplication protocol

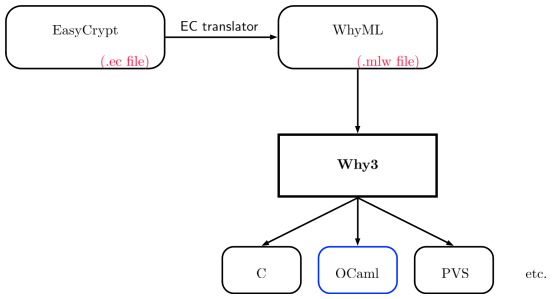

Figure 19: EasyCrypt extraction tool-chain. We only extract OCaml code in this work, but there are resources online suggesting that C and PVS code could also be extracted from WhyML.

process between the two platforms. Second, Why3 incorporates a powerful, verified, extraction mechanism, supporting extraction to multiple platforms and languages. However, some are not as mature (or verified) as the OCaml extraction. We view our tool-chain as an interesting starting point for a future (more general) EasyCrypt extraction tool-chain that makes use of a refined Why3 based code generation with support for multiple target languages.

From EasyCrypt to OCaml. We provide here an example illustration of our tool-chain using gradual secret sharing. We use the (proactive) gradual secret sharing scheme presented in [30, 33] to demonstrate how executable code can be obtained

from an EasyCrypt protocol specification. Briefly, the gradual secret sharing scheme is a composition of an additive secret sharing scheme and a *batch* secret sharing scheme. The additive secret sharing scheme is first executed using a secret s with the purpose of obtaining d summands $s_1, \ldots, s_d$ adding up to s. Then, every summand is shared linearly, increasing the degree of the sharing polynomial as parties advance on the summand they are sharing. We start by showing how the gradual secret sharing scheme was specified in EasyCrypt in Figure 20, where we assume that the scheme was instantiated for 15 parties and also assume the existence of an additive (ASS) and a *batch* (BSS) secret sharing schemes. Concrete EasyCrypt implementations of these secret sharing schemes can be found in [34].

```
theory GradualSecretSharingScheme.
  const n = 15.
  op k = BSS.k.
  op d = BSS.d.
  op t = BSS.t.
  type secret_t = ASS.secret_t.
  type share_t = BSS.share_t.
  type rand_t = ASS.rand_t * BSS.rand_t.
  op share (r : rand_t) (s : secret_t) : (BSS.p_id_t * share_t)
        list =
    let (ra,rb) = r in
    let summands = ASS.share ra s in
    BSS.share rb (map snd summands).
  op reconstruct (ss : (BSS.p_id_t * share_t) list) : secret_t
       option =
    let summands = BSS.reconstruct ss in
    ASS.reconstruct (zip ASS.p_id_set (oget summands)).
  clone import SecretSharingScheme as GradualSS with
    ...
end GradualSecretSharingScheme.
```

Figure 20: Gradual secret sharing scheme

We translate every type definition and functional operator to its counterpart in WhyML. We will focus only on the types and operators defined by GradualSS and omit definitions of dependencies such as list operations. The WhyML code generated for the same scheme can be found in Figure 21. As Figures 20 and 21 show, the code in the two scripts is very similar. Abstract values (both abstract operators and abstract constants) are defined in Why3 using `val`, every EasyCrypt **op** is mapped to an WhyML `let` and the *type* keyword is the same in both languages. Note, however, that the order of parametric types is reversed in both languages. In general, our translation tool performs a simple syntactic translation between the two languages, which increases our confidence about correctness of the tool even without a formal proof for this part.

The WhyML program is now ready for extraction. Applying the Why3 extraction mechanism yields the correct-by-construction OCaml code in Figure 22.

5 VERIFIED (PROACTIVE) SECRET SHARING AND MPC IMPLEMENTATION

Our verified implementation of the secure arithmetic evaluator was obtained via the EasyCrypt extraction tool described in Section 4, except the abstract libraries for randomness generation, Reed-Solomon codes and the cyclic group and finite

```
module GradualSS
  ...
  val n : int
  let k = 1
  let d = n - 3
  let t = d - 1
  type p_id_t = int
  val p_id_set : t list
  type secret_t = t
  type share_t = list  (t * (t * (list (int * group))))
  type rand_t = list t * (list (monomial list))
  let share (r : rand_t) (s : secret_t) : list (BSS.p_id_t *
      share_t)=
    let (ra,rb) = r in
    let summands = share ra s in
    share rb (map snd summands)
  let reconstruct (ss : list (p_id_t * share_t)) : option
      secret_t =
    let summands = reconstruct ss in
    reconstruct (zip p_id_set1 (oget summands))
```

Figure 21: Gradual secret sharing scheme in WhyML

```
let d1 : Z.t = Z.sub (Z.of_string "5") (Z.of_string "3")
let k : Z.t = d
let t : Z.t = Z.sub d Z.one
type p_id_t = Z.t
type secret_t = Z.t
type share_t = ((Z.t) * ((Z.t) * (Z.t * (Z.t list))) list) list
type rand_t = ((Z.t) list) * ((monomial list) list)
let share2 (r : ((Z.t) list) * ((monomial list) list)) (s : (Z.
      t)) : ((Z.t) * ((Z.t) * (Z.t * (Z.t list))) list) list =
  let (ra,rb) = r in
  let summands = share ra s in
  share1 rb (map snd summands)
let reconstruct2 (ss : ((Z.t) * ((Z.t) * (Z.t * (Z.t list)))
      list) list) : (Z.t) option =
  let summands = reconstruct ss in
  reconstruct1 (zip p_id_set1 (oget summands))
```

Figure 22: Gradual secret sharing scheme in OCaml

field structures, which were implemented using the CryptoKit library [10]. We generated cyclic group (and finite field) parameters using openssl and used ocaml-reed-solomon-erasure [11] for instantiating the Reed-Solomon error correction.

In this work we only focus on extracting an OCaml implementation (as highlighted in Figure 19). Because there are no publicly available implementations of proactive secret sharing and PMPC protocols, we compare the execution time of our extracted gradual secret sharing scheme with an unverified manual implementation of gradual secret sharing that we develop using the native secret sharing class in Charm [12]. We provide microbenchmarks for our extracted implementation in both the passive and malicious cases.

Benchmarking results of the secret sharing schemes are presented in Table 3, while benchmarks of the MPC protocols are shown in Table 4. We provide individual performance times for share, reconstruct, add, mult, refresh and recover for a field size of 128, 256, 512 and 1024 bits and for 5, 9 and 15 parties. We present the execution times in milliseconds. We

[10] https://github.com/xavierleroy/cryptokit
[11] https://gitlab.com/darrenldl/ocaml-reed-solomon-erasure
[12] https://github.com/JHUISI/charm

also provide an individual comparison with the Charm unverified implementation of the gradual secret sharing scheme in Table 2.

Our measurements were conducted on an x86-64 Intel Core i5 clocked at 2.4GHz with 256KB L2 cache per core. The extracted code was compiled with ocamlopt version 4.05.0 and the tests were ran in isolation, using the OCamlSys.time operator to read the execution time. We ran tests in batches of 100 runs each, noting the median of the times recorded in the runs.

Table 2: Performance benchmark table for extracted gradual secret sharing and for Charm unverified implementation for a 512 bit-size field.

		Share	Reconstruct
Charm	$n = 5$	6	0.3
	$n = 15$	23	2
Extracted	$n = 5$	0.19	0.6
	$n = 15$	1.5	6

We found that the sharing operation is faster in our generated code while reconstruct performs better in the Charm version. This may be justified by the verification penalty induced by the verified polynomial library (specially the polynomial interpolation) that only manifests itself in the reconstruction phase. In fact, the share protocol does not make use of significant polynomial operations (besides polynomial evaluation that does not represent a major overhead in the execution time) and so its performance depends on the performance of cyclic group and finite field operations. Since these are implemented with the Zarith library [13] (an OCaml wrapper for GMP), we are able to achieve a good performance for it. This is not the case for the reconstruction protocol, whose performance is greatly influenced by the performance of the polynomial library. By relying on a unverified library, Charm delivers better results for polynomial operations, thus delivering faster times for the reconstruct protocol.

Table 3: Performance benchmark table for extracted secret sharing implementation (times in *ms*). First column is the field size, the second column is the number of parties

		Shamir				Pedersen	
		Passive		Malicious		Unforgeable	
		Share	Reconstruct	Share	Reconstruct	Commit	Verify
128	5	0.02	0.05	0.15	0.02	0.06	0.02
	9	0.05	0.21	0.47	0.03	0.09	0.03
	15	0.12	0.58	1.21	0.04	0.16	0.05
256	5	0.02	0.10	0.57	0.09	0.25	0.06
	9	0.10	0.44	1.70	0.12	0.44	0.09
	15	0.19	0.98	4.81	0.18	0.64	0.11
512	5	0.03	0.17	2.27	0.35	1.24	0.24
	9	0.13	0.72	7.31	0.53	1.75	0.23
	15	0.26	1.70	20.36	0.79	2.81	0.29
1024	5	0.06	0.39	15.07	2.38	7.19	1.26
	9	0.19	1.29	48.92	3.60	12.08	1.34
	15	0.60	3.68	135.17	5.38	19.27	1.54

[13] https://github.com/ocaml/Zarith

Comparing with other optimized MPC implementations. A comparison with other (unverified) optimized implementations of MPC protocols, like EMP or SCALE-MAMBA, would be interesting but is outside the scope of this paper. *We stress that we do not claim to have the fastest implementation of a (proactive) secure MPC evaluator, nor is that our goal. Our goal is to demonstrate feasibility to performing computer-aided verification of such complex MPC protocols for active adversaries, and to automatically extract verified executable implementations thereof.* It would be interesting to explore how verified implementations behave in real software engineering projects, how the performance penalty induced by the verification process affects the overall performance of the system, and how much execution time developers are willing to sacrifice in order to have a more reliable system. Our performance comparison with Charm show that code obtained using our extraction approach is at least comparable with some manually implemented software, which is promising evidence that the verification overhead induced by our approach may not be as prohibitive as one may initially think. Likewise, the performance penalty of our solution is not intrinsic to the verification/extraction methodology. Some of the implementations cited above rely either on cryptographic optimizations, other (faster) protocols, faster underlying libraries (such as faster polynomial libraries) or some circuit optimization techniques. We point out that our overall verification approach can accommodate these cryptographic advances, with some implications on the security and correctness proofs. Exploring these issues is a promising avenue for future work as discussed next.

6 RELATED WORK

Previous computer-aided verification attempts paved the way for our work by demonstrating that verification of multiparty protocols is possible in EasyCrypt (even if only for smaller number of parties and/or for weaker adversaries).

The most relevant related work to ours in formal verification and high-assurance implementations of secure computation protocols is summarized in table 1. Staughton and Varia's work [66] only verifies one function-specific three party secure protocol that counts the number of elements in a database. Their work does not deal with a generic secure computation protocols, nor with more than 3 parties. Haagh, et al' [44] verify a simple MPC protocol due to Maurer [56], but does not synthesize a high-assurance (or certified in the terminology of [4]) software to evaluate the MPC protocol. In addition, the security proof in [44] is not completely (end-to-end) formalized and checked in EasyCrypt, instead only a proof that certain properties are satisfied by the protocol of [56], then it is manually proven that a simulator exists if these properties hold. Our work relies on an end-to-end proof in EasyCrypt without any manual steps. Finally, the verified version of Maurer's protocol in [44] is designed for general adversary structures and incurs exponential overhead (in size of shared data) in the threshold case. This limits [44] to a small number of parties.

The closest work to ours is that of Almeida, et al [4]. It develops a verified software stack for secure function evaluation by two parties in the semi-honest model. Their stack

Table 4: Performance benchmark table for extracted (P)MPC implementation (times in *ms*). First column is the field size, the second is the number of parties.

		Addition				Multiplication				Refresh				Recover			
		Passive		Malicious		Passive		Malicious		Passive		Malicious		Passive		Malicious	
		Total	Party	Total	Party	Total	Party	Total	Party	Total	Party	Total	Party	Total	Party	Total	Party
128	5	0.003	0.0006	0.007	0.001	0.08	0.02	1.64	0.33	0.08	0.02	1.23	0.25	0.12	0.02	0.99	0.20
	9	0.008	0.0009	0.01	0.0015	0.42	0.05	8.05	0.89	0.40	0.04	6.18	0.69	0.68	0.08	5.65	0.623
	15	0.02	0.0011	0.04	0.002	1.89	0.13	35.35	2.36	1.84	0.12	27.91	1.86	2.98	0.20	26.71	1.78
256	5	0.004	0.0009	0.01	0.001	0.11	0.02	6.48	1.30	0.10	0.02	4.60	0.92	0.17	0.03	3.61	0.72
	9	0.01	0.0007	0.02	0.002	0.60	0.07	33.83	3.76	0.57	0.06	24.89	2.77	1.06	0.12	22.12	2.46
	15	0.02	0.0011	0.05	0.003	2.79	0.19	148.13	9.88	2.66	0.18	110.53	7.37	4.60	0.31	103.52	6.90
512	5	0.004	0.0008	0.01	0.003	0.14	0.03	28.55	5.71	0.12	0.02	19.90	3.98	0.28	0.06	15.26	3.05
	9	0.01	0.0007	0.02	0.003	0.38	0.05	73.55	10.51	0.36	0.05	52.02	7.43	0.79	0.11	43.02	6.15
	15	0.02	0.0010	0.08	0.01	3.92	0.26	650.21	43.35	3.75	0.25	478.25	31.88	7.15	0.48	440.04	29.34
1024	5	0.004	0.0008	0.02	0.005	0.23	0.05	193.51	38.70	0.19	0.04	134.29	26.86	0.54	0.11	101.61	20.32
	9	0.01	0.0009	0.07	0.007	1.46	0.16	1015.89	112.88	1.36	0.15	725.83	80.65	3.11	0.35	625.88	69.54
	15	0.02	0.0011	0.14	0.01	7.64	0.51	4393.13	292.88	7.41	0.49	3202.29	213.49	14.17	0.94	2934.79	195.65

cannot handle more than two parties, nor active adversaries. The stack consists of two components: a certified compiler from C to Boolean circuits, and a high-assurance garbled circuit evaluator. The compiler proves that the output circuit is equivalent to the input C code. The Boolean circuit is then executed by a verified garbled circuit evaluator, synthesized from a formally verified EasyCrypt implementation of Yao's protocol.

In addition to the above, there have been impressive advances in research developing (practical) secure computation frameworks and software, either for two parties (typically based on garbled circuits) [55, 67], or for multiparty using algebraic MPC approach on top of secret sharing [14, 21, 68], or function-specific protocols based on (fully) homomorphic encryption, or mixed versions [29, 46]. Such frameworks are not directly comparable to our work because they do not claim to produce implementations of mechanically formally verified protocols nor executables thereof. We think that a very interesting future research direction is to perform computer-aided formal verification of the optimized protocols in the above frameworks and to attempt to mechanically synthesize efficient implementations thereof. We also hope that such mechanically synthesized implementations will have comparable performance to manually optimized ones.

7 CONCLUSIONS AND FUTURE WORK

Secure and privacy-preserving protocols are an example of advanced cryptographic constructions that will probably play a crucial role in the future of our networked world and the Internet. Computer-aided verification and automated software synthesis of such complicated protocols (e.g., with proactive security guarantees against active adversaries) is achievable with EasyCrypt as our work demonstrates. As a side contribution we perform the first computer-aided verification and automated synthesis of a variant of the (fundamental) BGW secure multiparty computation (MPC) protocol for static active adversaries. We also develop a tool-chain to verifiably extract executable implementations from EasyCrypt specifications of such protocols.

Future research directions include: 1) extending our work to provide stronger security guarantees and handle more settings, e.g., adaptive active adversaries as opposed to static ones we have for BGW, dealing with dynamic groups in the standard and proactive settings, and operating over asynchronous networks, 2) Performing computer-aided verification and synthesis of other (practical) MPC protocols such as SPDZ and its variations which is the basis for the efficient SCALE-MAMBA [14] MPC framework, or classic ones such as GMW [43] to deal with dishonest majorities, 3) Performing full verification and synthesis of UC-secure MPC protocols and primitives to enable arbitrary compositions of them and their executables. 4) Extending our protocol executables with verified libraries providing lower-level cryptographic algorithms and primitives similar to EverCrypt [15]. 5) Developing formal specification and computer-aided verifications of the underlying broadcast synchronous (or even asynchronous) communication.

ACKNOWLEDGMENTS

Vitor Pereira was supported by grant FCT-PD/BD/113967/201 awarded by Fundação para a Ciência e Tecnologia.

REFERENCES

[1] Masayuki Abe and Serge Fehr. 2004. Adaptively Secure Feldman VSS and Applications to Universally-Composable Threshold Cryptography. In *Advances in Cryptology – CRYPTO 2004*, Matt Franklin (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 317–334.

[2] Joseph A. Akinyele, Christina Garman, Ian Miers, Matthew W. Pagano, Michael Rushanan, Matthew Green, and Aviel D. Rubin. 2013. Charm: a framework for rapidly prototyping cryptosystems. *Journal of Cryptographic Engineering* 3, 2 (01 Jun 2013), 111–128. https://doi.org/10.1007/s13389-013-0057-3

[3] JosÃ© Bacelar Almeida, Manuel Barbosa, Gilles Barthe, and François Dupressoir. 2016. Verifiable side-channel security of cryptographic implementations: constant-time MEE-CBC. In *23rd International Conference on Fast Software Encryption (FSE)*. 163–184.

[4] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, and Vitor Pereira. 2017. A fast and verified software stack for secure function evaluation. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 1989–2006.

[5] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Hugo Pacheco, Vitor Pereira, and Bernardo Portela. 2018. Enforcing ideal-world leakage bounds in real-world secret sharing MPC frameworks. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. IEEE, 132–146.

[6] Gilad Asharov and Yehuda Lindell. 2011. A Full Proof of the BGW Protocol for Perfectly-Secure Multiparty Computation. Cryptology ePrint Archive, Report 2011/136. https://eprint.iacr.org/2011/136.

[7] Michael Backes, Aniket Kate, and Arpita Patra. 2011. Computational Verifiable Secret Sharing Revisited. In *Advances in Cryptology – ASIACRYPT 2011*, Dong Hoon Lee and Xiaoyun Wang (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 590–609.

[14] https://github.com/KULeuven-COSIC/SCALE-MAMBA

[15] https://project-everest.github.io

[8] Michael Backes, Matteo Maffei, and Esfandiar Mohammadi. 2010. Computationally Sound Abstraction and Verification of Secure Multi-Party Computations. In *FSTTCS*.

[9] Cécile Baritel-Ruet, François Dupressoir, Pierre-Alain Fouque, and Benjamin Grégoire. 2018. Formal Security Proof of CMAC and its Variants. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. IEEE, 91–104.

[10] Joshua Baron, Karim Eldefrawy, Joshua Lampkins, and Rafail Ostrovsky. 2014. How to withstand mobile virus attacks, revisited. In *PODC*. ACM, 293–302.

[11] Joshua Baron, Karim Eldefrawy, Joshua Lampkins, and Rafail Ostrovsky. 2015. Communication-Optimal Proactive Secret Sharing for Dynamic Groups. In *Applied Cryptography and Network Security*, Tal Malkin, Vladimir Kolesnikov, Allison Bishop Lewko, and Michalis Polychronakis (Eds.). Springer International Publishing, Cham, 23–41.

[12] Joshua Baron, Karim Eldefrawy, Joshua Lampkins, and Rafail Ostrovsky. 2015. Communication-Optimal Proactive Secret Sharing for Dynamic Groups. In *ACNS (LNCS)*, Vol. 9092. Springer, 23–41.

[13] Gilles Barthe, Cédric Fournet, Benjamin Grégoire, Pierre-Yves Strub, Nikhil Swamy, and Santiago Zanella-Béguelin. 2014. Probabilistic Relational Verification for Cryptographic Implementations. In *POPL*. To appear.

[14] Assaf Ben-David, Noam Nisan, and Benny Pinkas. 2008. FairplayMP: a system for secure multi-party computation. In *Proceedings of the 2008 ACM Conference on Computer and Communications Security, CCS 2008, Alexandria, Virginia, USA, October 27-31, 2008*, Peng Ning, Paul F. Syverson, and Somesh Jha (Eds.). ACM, 257–266. https://doi.org/10.1145/1455770.1455804

[15] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. 1988. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proceedings of the 20th Annual Symposium on Theory of Computing*. ACM, 1–10.

[16] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. 1988. Completeness Theorems for Non-Cryptographic Fault-Tolerant Distributed Computation (Extended Abstract). In *STOC*. ACM, 1–10.

[17] Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, Jianyang Pan, Jonathan Protzenko, Aseem Rastogi, Nikhil Swamy, Santiago Zanella-Béguelin, and Jean Karim Zinzindohoué. 2016. Implementing and Proving the TLS 1.3 Record Layer. Cryptology ePrint Archive, Report 2016/1178. http://eprint.iacr.org/2016/1178.

[18] Karthikeyan Bhargavan, Cédric Fournet, Markulf Kohlweiss, Alfredo Pironti, and Pierre-Yves Strub. 2013. Implementing TLS with Verified Cryptographic Security. In *IEEE S&P*.

[19] G. R. Blakley. 1979. Safeguarding cryptographic keys. *Proc. of AFIPS National Computer Conference* 48 (1979), 313–317.

[20] Dan Bogdanov, Peeter Laud, Sven Laur, and Pille Pullonen. 2014. From input private to universally composable secure multi-party computation primitives. In *Proceedings of the 27th Computer Security Foundations Symposium*. IEEE, 184–198.

[21] Dan Bogdanov, Sven Laur, and Jan Willemson. 2008. Sharemind: A Framework for Fast Privacy-Preserving Computations. In *ESORICS (Lecture Notes in Computer Science)*, Vol. 5283. Springer, 192–206.

[22] Sally Browning and Philip Weaver. 2010. *Designing Tunable, Verifiable Cryptographic Hardware Using Cryptol*. 89–143. https://doi.org/10.1007/978-1-4419-1539-9_4

[23] Christian Cachin, Klaus Kursawe, Anna Lysyanskaya, and Reto Strobl. 2002. Asynchronous verifiable secret sharing and proactive cryptosystems. In *ACM Conference on Computer and Communications Security*. 88–97.

[24] Christian Cachin, Klaus Kursawe, Anna Lysyanskaya, and Reto Strobl. 2002. Asynchronous Verifiable Secret Sharing and Proactive Cryptosystems. In *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS '02)*. ACM, New York, NY, USA, 88–97. https://doi.org/10.1145/586110.586124

[25] B. Chor, S. Goldwasser, S. Micali, and B. Awerbuch. 1985. Verifiable secret sharing and achieving simultaneity in the presence of faults. In *26th Annual Symposium on Foundations of Computer Science (sfcs 1985)*. 383–395. https://doi.org/10.1109/SFCS.1985.64

[26] Véronique Cortier, Constantin Catalin Dragan, François Dupressoir, and Bogdan Warinschi. 2018. Machine-checked proofs for electronic voting: privacy and verifiability for Belenios. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. IEEE, 298–312.

[27] Morten Dahl and Ivan Damgård. 2014. Universally Composable Symbolic Analysis for Two-Party Protocols Based on Homomorphic Encryption. In *EUROCRYPT*.

[28] Ivan Damgård, Kasper Damgård, Kurt Nielsen, Peter Sebastian Nordholt, and Tomas Toft. 2016. Confidential Benchmarking based on Multiparty Computation. In *Proceedings of the 20th International Conference on Financial Cryptography and Data Security*. Springer, 169–187.

[29] Daniel Demmler, Thomas Schneider, and Michael Zohner. 2015. ABY - A Framework for Efficient Mixed-Protocol Secure Two-Party Computation. In *22nd Annual Network and Distributed System Security Symposium, NDSS 2015, San Diego, California, USA, February 8-11, 2015*. https://www.ndss-symposium.org/ndss2015/aby---framework-efficient-mixed-protocol-secure-two-party-computation

[30] Shlomi Dolev, Karim Eldefrawy, Joshua Lampkins, Rafail Ostrovsky, and Moti Yung. 2016. Proactive Secret Sharing with a Dishonest Majority. In *Security and Cryptography for Networks*, Vassilis Zikas and Roberto De Prisco (Eds.). Springer International Publishing, Cham, 529–548.

[31] Shlomi Dolev, Karim Eldefrawy, Joshua Lampkins, Rafail Ostrovsky, and Moti Yung. 2016. Proactive Secret Sharing with a Dishonest Majority. In *SCN (LNCS)*, Vol. 9841. Springer, 529–548.

[32] Yael Ejgenberg, Moriya Farbstein, Meital Levy, and Yehuda Lindell. 2012. SCAPI: The Secure Computation Application Programming Interface. Cryptology ePrint Archive, Report 2012/629. https://eprint.iacr.org/2012/629.

[33] Karim Eldefrawy, Rafail Ostrovsky, Sunoo Park, and Moti Yung. 2018. Proactive Secure Multiparty Computation with a Dishonest Majority. In *Proceedings of the Eleventh Conference on Security and Cryptography for Networks*. 200–215. https://doi.org/10.1007/978-3-319-98113-0_11

[34] Karim Eldefrawy and Vitor Pereira. 2019. A High-Assurance Evaluator for Machine-Checked Secure Multiparty Computation. Cryptology ePrint Archive, Report 2019/922. https://eprint.iacr.org/2019/922.

[35] P. Feldman. 1987. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science (sfcs 1987)*. 427–438. https://doi.org/10.1109/SFCS.1987.4

[36] Jean-Christophe Filliâtre. 2013. One Logic To Use Them All. In *24th International Conference on Automated Deduction (CADE-24) (Lecture Notes in Artificial Intelligence)*, Vol. 7898. Springer, Lake Placid, USA, 1–20.

[37] Jean-Christophe Filliâtre and Andrei Paskevich. 2013. Why3 — Where Programs Meet Provers. In *Proceedings of the 22nd European Symposium on Programming (Lecture Notes in Computer Science)*, Matthias Felleisen and Philippa Gardner (Eds.), Vol. 7792. Springer, 125–128.

[38] Matthias Fitzi, Juan Garay, Shyamnath Gollakota, C. Pandu Rangan, and Kannan Srinathan. 2006. Round-Optimal and Efficient Verifiable Secret Sharing. In *Theory of Cryptography*, Shai Halevi and Tal Rabin (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 329–342.

[39] Cédric Fournet, Markulf Kohlweiss, and Pierre-Yves Strub. 2011. Modular code-based cryptographic verification. In *ACM CCS*.

[40] Matthew Franklin and Moti Yung. 1992. Communication Complexity of Secure Computation (Extended Abstract). In *Proceedings of the Twenty-fourth Annual ACM Symposium on Theory of Computing (STOC '92)*. ACM, New York, NY, USA, 699–710. https://doi.org/10.1145/129712.129780

[41] Rosario Gennaro, Yuval Ishai, Eyal Kushilevitz, and Tal Rabin. 2001. The Round Complexity of Verifiable Secret Sharing and Secure Multicast. In *Proceedings of the Thirty-third Annual ACM Symposium on Theory of Computing (STOC '01)*. ACM, New York, NY, USA, 580–589. https://doi.org/10.1145/380752.380853

[42] Rosario Gennaro, Michael O. Rabin, and Tal Rabin. 1998. Simplified VSS and Fast-track Multiparty Computations with Applications to Threshold Cryptography. In *Proceedings of the Seventeenth Annual ACM Symposium on Principles of Distributed Computing (PODC '98)*. ACM, New York, NY, USA, 101–111. https://doi.org/10.1145/277697.277716

[43] Oded Goldreich, Silvio Micali, and Avi Wigderson. 1987. How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority. In *STOC*. 218–229.

[44] Helene Haagh, Aleksandr Karbyshev, Sabine Oechsner, Bas Spitters, and Pierre-Yves Strub. 2018. Computer-Aided Proofs for Multiparty Computation with Active Security. In *31st IEEE Computer Security Foundations Symposium, CSF 2018, Oxford, United Kingdom, July 9-12, 2018*. IEEE Computer Society, 119–131. https://doi.org/10.1109/CSF.2018.00016

[45] Helene Haagh, Aleksandr Karbyshev, Sabine Oechsner, Bas Spitters, and Pierre-Yves Strub. 2018. Computer-Aided Proofs for Multiparty Computation with Active Security. 119–131. https://doi.org/10.1109/CSF.2018.00016

[46] Wilko Henecka, Stefan K ögl, Ahmad-Reza Sadeghi, Thomas Schneider, and Immo Wehrenberg. 2010. TASTY: Tool for Automating Secure Two-party Computations. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS '10)*. ACM, New York, NY, USA, 451–462. https://doi.org/10.1145/1866307.1866358

[47] Wilko Henecka, Stefan KÃ¶gl, Ahmad-Reza Sadeghi, Thomas Schneider, and Immo Wehrenberg. 2010. TASTY: Tool for Automating Secure Two-partY computations. *IACR Cryptology ePrint Archive* 2010, 451–462. https://doi.org/10.1145/1866307.1866358

[48] Amir Herzberg, Stanislaw Jarecki, Hugo Krawczyk, and Moti Yung. 1995. Proactive Secret Sharing Or: How to Cope With Perpetual Leakage. In *CRYPTO*. 339–352.

[49] Amir Herzberg, Stanisław Jarecki, Hugo Krawczyk, and Moti Yung. 1995. Proactive Secret Sharing Or: How to Cope With Perpetual Leakage. In *Advances in Cryptology — CRYPT0' 95*, Don Coppersmith (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 339–352.

[50] Martin Hirt, Christoph Lucas, and Ueli Maurer. 2013. A Dynamic Tradeoff between Active and Passive Corruptions in Secure Multi-Party Computation. In *CRYPTO (2) (LNCS)*, Vol. 8043. Springer, 203–219.

[51] Jonathan Katz, Chiu-Yuen Koo, and Ranjit Kumaresan. 2008. Improving the Round Complexity of VSS in Point-to-Point Networks. In *Automata, Languages and Programming*, Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfsdóttir, and Igor Walukiewicz (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 499–510.

[52] Hugo Krawczyk. 2001. The Order of Encryption and Authentication for Protecting Communications (or: How Secure Is SSL?). In *Advances in Cryptology — CRYPTO 2001*, Joe Kilian (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 310–331.

[53] Ranjit Kumaresan, Arpita Patra, and C. Pandu Rangan. 2010. The Round Complexity of Verifiable Secret Sharing: The Statistical Case. In *Advances in Cryptology - ASIACRYPT 2010*, Masayuki Abe (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 431–447.

[54] Baiyu Li and Daniele Micciancio. 2018. Symbolic Security of Garbled Circuits. 147–161. https://doi.org/10.1109/CSF.2018.00018

[55] Dahlia Malkhi, Noam Nisan, Benny Pinkas, and Yaron Sella. 2004. Fairplay - Secure Two-Party Computation System. In *Proceedings of the 13th USENIX Security Symposium, August 9-13, 2004, San Diego, CA, USA*, Matt Blaze (Ed.). USENIX, 287–302. http://www.usenix.org/publications/library/proceedings/sec04/tech/malkhi.html

[56] Ueli M. Maurer. 2006. Secure multi-party computation made simple. *Discrete Applied Mathematics* 154, 2 (2006), 370–381. https://doi.org/10.1016/j.dam.2005.03.020

[57] Rafail Ostrovsky and Moti Yung. 1991. How to Withstand Mobile Virus Attacks (Extended Abstract). In *PODC*. ACM, 51–59.

[58] Mário José Parreira Pereira. 2018. *Tools and Techniques for the Verification of Modular Stateful Code.* Ph.D. Dissertation. Paris Saclay.

[59] Arpita Patra, Ashish Choudhary, Tal Rabin, and C. Pandu Rangan. 2009. The Round Complexity of Verifiable Secret Sharing Revisited. In *Advances in Cryptology - CRYPTO 2009*, Shai Halevi (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 487–504.

[60] Torben Pryds Pedersen. 1991. A Threshold Cryptosystem Without a Trusted Party. In *Proceedings of the 10th Annual International Conference on Theory and Application of Cryptographic Techniques (EUROCRYPT'91)*. Springer-Verlag, Berlin, Heidelberg, 522–526. http://dl.acm.org/citation.cfm?id=1754868.1754929

[61] Torben Pryds Pedersen. 1992. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *Advances in Cryptology — CRYPTO '91*, Joan Feigenbaum (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 129–140.

[62] Aseem Rastogi, Nikhil Swamy, and Michael Hicks. 2017. WYS*: A Verified Language Extension for Secure Multi-party Computations. *CoRR* abs/1711.06467 (2017). arXiv:1711.06467 http://arxiv.org/abs/1711.06467

[63] David Schultz. 2007. *Mobile Proactive Secret Sharing.* Ph.D. Dissertation. Massachusetts Institute of Technology.

[64] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613.

[65] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (Nov. 1979), 612–613. https://doi.org/10.1145/359168.359176

[66] Alley Stoughton and Mayank Varia. 2017. Mechanizing the Proof of Adaptive, Information-Theoretic Security of Cryptographic Protocols in the Random Oracle Model. In *30th IEEE Computer Security Foundations Symposium, CSF 2017, Santa Barbara, CA, USA, August 21-25, 2017*. IEEE Computer Society, 83–99. https://doi.org/10.1109/CSF.2017.36

[67] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. 2017. Faster Secure Two-Party Computation in the Single-Execution Setting. In *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part III (Lecture Notes in Computer Science)*, Jean-Sébastien Coron and Jesper Buus Nielsen (Eds.), Vol. 10212. 399–424. https://doi.org/10.1007/978-3-319-56617-7_14

[68] Xiao Wang, Samuel Ranellucci, and Jonathan Katz. 2017. Global-Scale Secure Multiparty Computation. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM, 39–56. https://doi.org/10.1145/3133956.3133979

[69] Theodore M. Wong, Chenxi Wang, and Jeannette M. Wing. 2002. Verifiable Secret Redistribution for Archive System. In *IEEE Security in Storage Workshop*. IEEE Computer Society, 94–106.

[70] Andrew Chi-Chih Yao. 1982. Protocols for Secure Computations (Extended Abstract). In *FOCS*. IEEE Computer Society, 160–164.

[71] Lidong Zhou, Fred B. Schneider, and Robbert van Renesse. 2005. APSS: proactive secret sharing in asynchronous systems. *ACM Trans. Inf. Syst. Secur.* 8, 3 (2005), 259–286.

A REASONING ABOUT POLYNOMIALS

We had to formalize a verified library to reason about polynomials because several protocols rely heavily on polynomials and operations over them. This was a critical step in our proofs, otherwise we would have needed to use a non-verified polynomials library to perform the desired operations, which would increase our trusted code base.

We are interested in polynomials over finite fields in this work. However, we developed a general library, that could be reused to define multiple instances of polynomials. We started the development of the library by first defining required abstract definitions. These abstract definitions provide an interface for concrete realizations of polynomials and define the coefficient type, polynomial evaluation, the *zero* and *one* polynomials, polynomial degree, addition, multiplication, unary minus and interpolation. We fix the type of the polynomials to be a list of monomials, which induces a possible equality class to be the equality between lists. Such equality class is too strong and would force the definition of complicated operators to perform addition and multiplication, thus inducing a significant performance penalty. We fix the equality class to be the equality of the evaluation of two polynomials on the same points, and use this equality class to define the subsequent axioms and lemmas around the operations of polynomials mentioned above. This abstract interface for polynomials is reusable and can be applied to other domains.

We then instantiate the type of the coefficients to be the same type of elements in a finite field and define all polynomials operations in a natural way. Properties such as commutativity and associativity of addition and multiplication of polynomials were proven by relying on the same properties verified in finite field operations. We also provide a formalization for polynomial interpolation based on Lagrange interpolation, which makes use of a linear combination of Lagrange basis polynomials. Note that polynomial interpolation is important in our work, since it is used for secret reconstruction inside the reconstruct protocol and also by recovering parties to successfully recover their shares.

We define polynomials by relying on EasyCrypt's record system as shown in Figure 23. We view polynomials as lists of monomials, which are a record with two fields: a *coefficient* (an element of a finite field) and an *exponent* (an integer).

```
type coefficient = t.
type exponent = int.
type monomial = {
  coef : coefficient;
  expo : exponent
}.
type polynomial = monomial list.
```

Figure 23: Polynomial type definition

Next, our polynomials library defines evaluation functions, one for monomial evaluation and another for polynomials, the latter is basically multiple applications of the former. Evaluation of polynomials defines our equality class. These definitions can be found in Figure 24. We also include in this Figure

```
op madd m1 m2 = {| coef = m1.`coef + m2.`coef; expo = m1.`expo
    |}.
op mpadd (m : monomial) p =
  with p = [] => [m]
  with p = m' :: p' =>
    if m.`expo = m'.`expo then madd m m' :: p'
    else
      if m'.`expo < m.`expo then m :: p
      else m' :: mpadd m p'.
op add (p1 p2 : polynomial) =
  with p1 = [], p2 = [] => []
  with p1 = m1 :: p1', p2 = [] => p1
  with p1 = [], p2 = m2 :: p2' => p2
  with p1 = m1 :: p1', p2 = m2 :: p2' =>
    if m1.`expo = m2.`expo then madd m1 m2 :: add p1' p2'
    else
      if m1.`expo < m2.`expo then m2 :: add p1 p2'
      else m1 :: add p1' p2.
op mmul m1 m2 =
  if m1 = mzero \/ m2 = mzero then mzero
  else {| coef = m1.`coef * m2.`coef; expo = m1.`expo + m2.`
      expo |}.
op mpmul m p =
  with p = [] => []
  with p = m' :: p' => mpadd (mmul m m') (mpmul m p').
op mul p1 p2 =
  with p1 = [] => []
  with p1 = m :: p1' => add (mpmul m p2) (mul p1' p2).
op mumin m = {| coef = - m.`coef; expo = m.`expo |}.
op umin p =
   with p = [] => []
   with p = m :: p' => mumin m :: umin p'.
```

Figure 26: Polynomial arithmetic

a polynomial *membership* test: a point is on a polynomial if the evaluation of the polynomial at the point's abscissa is equal to the value of the point's ordinate.

```
op meval (x:coefficient) (m : monomial) = m.`coef * (x ^ m.`
    expo).
op eval (x:coefficient) p =
  with p = [] => F.zero
  with p = m :: p' => meval x m + (eval x p').
op (==) p1 p2 = forall x, eval x p1 = eval x p2.
op mem (pt : (coeficient * coeficient)) p = eval (fst pt) p = (
    snd pt).
```

Figure 24: Polynomial evaluation and equality class

We are also interested in having a *zero* (i.e., a polynomial that always evaluates to zero) and a *one* polynomial (i.e., a polynomial that always evaluate to one) so that we are able to define algebraic properties around operations using polynomials. These two polynomials are depicted in Figure 25. Note that `zero` could also be defined based on `mzero`; defining it as an empty list simplifies proofs because it allows one to prove properties of polynomials based on list induction.

```
op mzero = {| coef = F.zero; expo = 1 |}.
op zero : polynomial = [].
op mone = {| coef = F.one; expo = 0 |}.
op one = [mone].
```

Figure 25: *Zero* and *one* polynomials

Figure 26 sketches the EasyCrypt definition of arithmetic of polynomials. Every arithmetic operation is defined in the same, mechanical way: 1) we start by defining that operation in terms of monomials (`madd`, `mmul` and `mumin`); 2) then we define it in terms of a monomial and a polynomial (`mpadd` and `mpmul`); and finally 3) we define the polynomial operation based on the previous two (`add`, `mul` and `umin`). The reason why we implement polynomials operations as such is because it simplifies the proofs. Proving properties related to monomial operations is easier than proving them applied to polynomials. Yet, since polynomial operations are defined based on operations using monomials, it is easy to propagate results obtained at monomial level to polynomials. Additionally, interesting arithmetic properties (such as commutativity or associativity) of polynomial operations can be proven by relying on the same properties of their underlying coefficients.

The final operation required in our polynomials interface is that of the Lagrange interpolation. Lagrange interpolation allows one to reconstruct a $d-1$ degree polynomial based on d points on that polynomial. Briefly, it works by computing *bases* based on the abscissa values of the given d points, which are then multiplied by the ordinate values. As part of this work, we provide two different interpolation functions:

- `interpolate` - taking as input a set of points and some x value, returns the evaluation of the interpolated polynomial at x
- `interpolate_poly` - taking as input a set of points, returns the interpolated polynomial.

An illustration of our Lagrange polynomial interpolation in EasyCrypt is presented in Figure 27.

```
op basis_loop (x : t) (xmx : t) (xm : t list) : t list =
  with xm = [] => []
  with xm = y :: ys =>
    if y <> xmx then
      ((x - y) / (xmx - y)) :: basis_loop x xmx ys
    else basis_loop x xmx ys.
op basis (x xmx : t) (xm : t list) =
  foldr (fun (x y : t) => x * y) F.one (basis_loop x xmx xm).
op interpolate_loop (x : t) (xm : t list) (pm : (t * t) list) =
  with pm = [] => []
  with pm = y :: ys =>
    ((basis x (fst y) xm) * (snd y)) :: interpolate_loop x xm ys
        .
op interpolate (x : t) (pm : (t * t) list) =
  let xm = map fst pm in
  let bs = interpolate_loop x xm pm in
  foldr (fun (x y : t) => x + y) F.zero bs.
```

Figure 27: Lagrange's polynomial interpolation